

A Modular Framework for Guarded Autonomous Digital Twin Supervisory Control

Author Name(s) Omitted for Draft

Department / Laboratory / Institution

City, Country

email@example.com

Abstract—Digital twins are increasingly used to represent, monitor, and evaluate engineered systems, yet many deployments remain advisory rather than autonomous. Moving from passive digital twin visualization toward autonomous digital twin control requires more than connecting an artificial intelligence model to a simulator. A practical autonomous digital twin controller must define a stable state-action contract, separate high-level supervisory decisions from low-level plant control, enforce safety constraints outside the policy model, support multiple runtime backends, and produce an auditable record of each decision. This paper proposes a modular framework for guarded autonomous digital twin supervisory control. The framework treats the digital twin as an environment exposed through a compact observation interface and a bounded command interface. A supervisory runner coordinates observation, policy selection, command limiting, backend execution, safety checking, and mission reporting. Control policies are interchangeable and may include deterministic rule-based policies, learned policies, or large-language-model decision policies, while an independent guard layer enforces hard constraints, soft-constraint reporting, command clipping, rate limiting, and fallback behavior. The framework is demonstrated through a battery-assisted dc load-bus digital twin with MATLAB/Simulink and UDP runtime interfaces. The case study illustrates how the proposed architecture supports mock testing, live digital twin communication, policy substitution, and transparent decision logging. The result is a reusable design pattern for developing autonomous digital twin controllers that are inspectable, extensible, and constrained by explicit operational safety contracts.

Index Terms—Digital twin, autonomous control, supervisory control, guarded autonomy, large language models, Simulink, microgrids, dc power systems, safety constraints, decision logging.

I. INTRODUCTION

Digital twins have become an important mechanism for representing physical systems, integrating simulation models with operational data, and supporting engineering decision-making. In power and energy systems, manufacturing, transportation, and cyber-physical infrastructure, digital twins can provide a computational view of asset state, forecast future behavior, and evaluate hypothetical interventions before those interventions are applied to a real system. Despite this promise, many digital twin systems remain passive: they visualize state, answer questions, or provide offline analysis, but they do not participate directly in closed-loop or supervisory control.

Autonomous digital twin control is a natural next step, but it also creates a systems problem that is different from model construction alone. A digital twin controller must interact with

a model or live simulation, interpret state estimates, select actions, and apply those actions through a communication pathway. If artificial intelligence (AI) is used as part of the decision process, the architecture must also define what the AI is allowed to decide, how its outputs are validated, and how the system responds when an AI output is unsafe, malformed, delayed, or unavailable. In practical engineering environments, the controller must also remain auditable: operators and researchers should be able to inspect what the digital twin observed, what command was issued, why the command was selected, which constraints were active, and why the mission succeeded or failed.

These requirements suggest that autonomous digital twin control should not be treated as a direct connection between an AI model and a plant model. Instead, it should be treated as a guarded supervisory-control framework. The digital twin provides a structured observation. A policy proposes a bounded high-level command. A guard layer validates and limits the command. A backend adapter applies the command to a simulation, live model, or physical interface. A reporting layer records the complete decision trace. This architecture allows autonomy to be introduced incrementally while preserving clear responsibility boundaries between the policy, the digital twin, the safety logic, and the plant-level controllers.

This paper proposes such a framework. The framework is designed for autonomous digital twin supervisory control, where the autonomous agent is responsible for high-level operating decisions rather than direct low-level control signals. In the motivating case study, a digital twin of a battery-assisted dc load bus exposes state variables such as state of charge (SOC), bus voltage, battery power, load power, source power, source availability, and fault flags. The supervisory controller chooses from a bounded command set: charge, hold, discharge, or safe. The inner converter control remains inside the Simulink model, while the autonomous supervisor commands a battery-power reference. This separation prevents the autonomous policy from issuing raw actuator commands such as duty-ratio values.

The proposed framework is intentionally policy agnostic. A deterministic state-of-charge and bus-voltage policy can be used for baseline behavior. A learned or large-language-model (LLM) policy can be substituted through the same interface. In all cases, the runner owns hard safety stops, command clipping, rate limits, and mission logging. Thus, the frame-

work supports experimentation with advanced policy models without transferring safety responsibility to those models.

The main contributions of this paper are:

- a modular framework for autonomous digital twin supervisory control based on an explicit observation, goal, command, and backend contract;
- a guarded execution architecture that separates policy proposals from safety enforcement, command limiting, and mission termination;
- a policy interface that supports deterministic, learned, or LLM-based supervisory decisions without changing the digital twin backend;
- a backend abstraction that supports mock simulation, MATLAB/Simulink engine execution, and live UDP communication with a running digital twin; and
- a case-study implementation for a battery-assisted dc bus digital twin, including decision traces and mission-report generation for framework evaluation.

II. MOTIVATION AND REQUIREMENTS

A. Digital Twins as Runtime Control Contexts

A digital twin is commonly understood as a digital representation of a physical system that is linked to data, models, or operational context [1], [2]. In many engineering applications, the digital twin supports monitoring, diagnosis, simulation, and forecasting. Once the digital twin is connected to a control task, however, its role changes. It is no longer only an information source; it becomes part of a decision loop.

This shift introduces several architectural questions. What information does the twin expose to the controller? Which actions may the controller request? How are constraints represented? Does the digital twin own the plant-level controller, or does the autonomous agent command low-level actuator variables directly? How are decisions logged? How does the system behave when the communication path fails?

These questions are not solved by model fidelity alone. A high-fidelity digital twin can still be unsafe or difficult to control if its interface to the autonomous supervisor is unstructured. Conversely, a simplified digital twin can be useful for early autonomy research if it exposes a clear state-action contract and supports repeatable testing. The proposed framework therefore focuses on the interface layer between digital twin runtime and autonomous supervisory policy.

B. Supervisory Control Rather than Direct Actuation

The framework proposed here is designed for supervisory control. The autonomous digital twin controller selects high-level commands, operating modes, or references, while lower-level control loops remain responsible for fast plant dynamics. For a battery-connected dc bus, the supervisor may request a battery-power reference, but it should not directly command the converter duty ratio. The duty ratio, pulse-width modulation, current limiting, and converter dynamics remain inside the plant controller or simulation model.

This separation is particularly important when AI models are involved. A language model or learned policy may be

useful for selecting a mode, interpreting a fault context, or choosing a conservative power request. It should not be responsible for raw switching commands. The framework therefore constrains the policy action space to bounded supervisory commands and delegates fast physical control to the digital twin or embedded control layer.

C. Framework Requirements

The framework is organized around seven requirements:

- 1) *Explicit state-action contract*: the twin exposes typed observations and accepts bounded commands.
- 2) *Policy substitutability*: deterministic, learned, or LLM policies can be exchanged without changing the runner or backend.
- 3) *Safety ownership outside the policy*: hard constraints, command bounds, rate limits, and fallback behavior are enforced by the runner rather than by the policy alone.
- 4) *Backend independence*: the same supervisory loop can operate over mock simulation, step-based Simulink execution, or live UDP communication.
- 5) *Communication robustness*: stale observations, repeated commands, packet schemas, startup transients, and backend exceptions are handled by adapters and guards.
- 6) *Auditability*: each decision records the observation, selected command, command reason, status, and active violations.
- 7) *Incremental autonomy*: development can progress from deterministic policies and mock simulations toward AI policies and live digital twin communication.

These requirements position the contribution as a framework paper rather than as a claim that a particular controller is globally optimal. The objective is to define and demonstrate an architecture that makes autonomous digital twin supervisory control modular, inspectable, and safe enough for systematic experimentation.

III. PROPOSED FRAMEWORK

A. Architecture Overview

Fig. 1 summarizes the proposed architecture. The framework consists of six components: a digital twin backend, a structured observation object, a goal object, a policy module, a guarded runner, and a reporting layer. At each supervisor tick, the runner receives or requests an observation from the backend. It evaluates hard and soft constraints. If a hard constraint is violated, the runner sends a safe command and terminates the mission. If the mission has satisfied its success criteria for the required duration, the runner returns success. Otherwise, it calls the active policy, limits the resulting command, applies the command through the backend, records the event, and advances to the next observation.

B. Formal Representation

Let the digital twin at supervisory step k expose an observation

$$o_k = O(DT, t_k), \quad (1)$$

Digital Twin Backend $\rightarrow$ Observation Contract $\rightarrow$ Guarded Runner $\rightarrow$ Policy Interface $\rightarrow$ Command Limiter $\rightarrow$
Backend Command Path $\rightarrow$ Mission Log

Backends: mock simulator, MATLAB/Simulink engine bridge, live UDP interface. Policies: deterministic rule policy, learned policy, LLM policy. Guards: hard stops, soft-violation logging, command clipping, rate limits, fallback behavior.

Fig. 1. Proposed guarded autonomous digital twin supervisory-control framework. The policy proposes high-level commands, while the runner enforces safety and execution constraints before commands reach the digital twin backend.

where O is the backend observation function, DT is the digital twin runtime, and t_k is the current simulation or wall-clock time. Let the mission goal be

$$G = \{B_{\text{soft}}, B_{\text{hard}}, T_{\text{success}}, T_{\text{max}}\}, \quad (2)$$

where B_{soft} defines desired operating bands, B_{hard} defines hard safety limits, T_{success} defines the required stable hold time, and T_{max} defines the maximum mission duration.

A policy proposes a command:

$$u_k^p = \pi(o_k, G). \quad (3)$$

The guard layer maps the proposed command to an executable command:

$$u_k = \gamma(u_k^p, o_k, G, u_{k-1}), \quad (4)$$

where γ enforces safety stops, command bounds, rate limits, enable flags, and fallback behavior. The backend then applies the command:

$$DT_{k+1} = A(DT_k, u_k, \Delta t), \quad (5)$$

where A is implemented by a simulator, a MATLAB/Simulink bridge, a live UDP interface, or another runtime adapter. Thus, the policy selects u_k^p , while the guard determines whether and how it becomes u_k .

C. Component Interfaces

The backend interface contains five operations:

$$\begin{aligned} \text{reset}() \rightarrow o, \quad \text{observe}() \rightarrow o, \\ \text{apply_command}(u), \quad \text{advance}(\Delta t) \rightarrow o, \quad \text{close}(). \end{aligned} \quad (6)$$

This interface is intentionally small. It is sufficient for mock simulation, step-based simulator integration, and live communication. In live UDP mode, the `advance` operation does not force the simulator to step; instead, it waits until the live digital twin publishes an observation whose simulation timestamp has advanced sufficiently.

The policy interface contains one operation:

$$\text{choose_action}(o, G) \rightarrow u^p. \quad (7)$$

This allows a deterministic controller, optimization routine, reinforcement-learning policy, or language-model policy to be substituted without modifying the runner.

Guarded supervisory loop

- 1) Reset backend and observe digital twin.
- 2) Check hard constraints; issue `SAFE` on violation.
- 3) Record soft violations.
- 4) If goal is held for the required duration, return success.
- 5) During startup grace, hold. Otherwise, call policy.
- 6) Clip and rate-limit command.
- 7) Apply command, log event, and advance to next observation.
- 8) Stop on success, safety stop, timeout, backend failure, or strict policy failure.

Fig. 2. Compact algorithmic view of the guarded autonomous supervisory loop.

IV. CONTRACTS AND GUARDED EXECUTION

A. Observation, Command, and Goal Contracts

The framework uses explicit contracts for observations, commands, and mission goals. Table I summarizes the case-study contract. The observation is not intended to contain every simulator variable; it exposes the variables required for supervisory decision-making and safety monitoring. Additional internal variables remain inside the digital twin model.

For the current case study, the target SOC band is 40–80%, the SOC hard limits are 10–95%, the nominal bus voltage is 400 V, the bus soft band is $\pm 5\%$ of nominal, and the bus hard band is $\pm 20\%$ of nominal. These values are configurable; they are included here to make the case study concrete.

B. Guarded Runner Logic

The runner enforces safety and validity checks that are deliberately outside the policy:

- a startup grace period for tolerating simulator initialization transients;
- hard-constraint checking over SOC, bus voltage, optional current limits, and fault flags;
- soft-constraint reporting for target-band violations;
- command clipping against a maximum battery-power magnitude;
- command rate limiting between supervisor ticks;
- disabled-command behavior that resets the held power command to zero; and
- fallback behavior for AI policy failures.

These mechanisms make the policy a proposal generator rather than the final execution authority.

C. Mission Outcomes

The runner returns one of five statuses: `running`, `succeeded`, `timed_out`, `safety_stopped`, or

TABLE I
STATE-ACTION-GOAL CONTRACT USED IN THE BATTERY-ASSISTED DC BUS CASE STUDY.

Contract	Fields	Purpose
Observation	sim_time_s, soc_pct, v_bus_v, v_batt_v, i_batt_a, p_batt_w, p_load_w, p_source_w, source flags, fault flags	Expose the compact digital twin state needed for supervisory decisions, constraint checks, logging, and backend-independent policy execution.
Command	mode, p_batt_cmd_w, enable, reason	Restrict the autonomous supervisor to high-level commands: CHARGE, HOLD, DISCHARGE, or SAFE. Positive battery power discharges into the bus; negative power charges the battery.
Goal	SOC target band, SOC hard limits, bus nominal voltage, bus soft band, bus hard band, success hold time, maximum mission time, optional current limit	Separate desired operating regions from hard safety constraints and mission termination criteria.

```
{
  "mode": "CHARGE | HOLD | DISCHARGE | SAFE",
  "p_batt_cmd_w": 0.0,
  "reason": "short explanation"
}
```

Fig. 3. Required response schema for an LLM-based supervisory policy.

failed. This status set distinguishes control performance, safety termination, infrastructure failure, and incomplete missions. Each decision event stores the tick index, observation, command, active violations, and status. This design supports later analysis without requiring the researcher to reconstruct decisions from console output.

V. POLICY AND GUARD LAYER

A. Deterministic Baseline Policy

The baseline policy is a deterministic SOC and bus-voltage band policy. It first checks active fault flags. If faults are active, it requests a safe command. If bus voltage is below the soft lower limit and the battery is above the hard reserve, it requests battery discharge to support the bus. If bus voltage is above the soft upper limit and the battery is not near the upper target, it requests charging to absorb power. If SOC is below the target band and the source is available, it requests charging. If SOC is above the target band, it requests discharge. Otherwise, it holds.

This policy is intentionally simple. Its role is not to represent a final optimal controller. Its role is to provide a transparent baseline and fallback policy for framework development.

B. AI and LLM Policy Interface

The same policy interface can be implemented by an AI policy. In the current implementation, an OpenAI-compatible chat-completion client can be used as a high-level supervisory policy. The model receives a structured JSON payload containing the goal, observation, and required response schema. It must return one compact JSON object:

The language model is not allowed to command duty ratio, switching states, or direct actuator values. It only proposes the high-level supervisory command. The runner parses the JSON,

TABLE II
BACKEND MODES SUPPORTED BY THE FRAMEWORK.

Backend	Role
Mock battery-bus simulator	Deterministic approximation for runner, policy, guard, and report testing.
MATLAB/Simulink bridge	Step-based execution that initializes the model, applies command variables, steps simulation time, and extracts observations.
Live UDP backend	Interface to a continuously running Simulink model that publishes observations and receives supervisory command packets.

converts it to the command object, clips the command, applies rate limits, and handles failures.

If the AI policy fails to produce valid JSON or if the model call fails, the framework can fall back to the deterministic baseline policy. In strict experimental mode, the same failure can instead terminate the run and record the policy failure. This option separates control robustness experiments from AI-output-validity experiments.

C. Trace Logging

For AI policies, the framework can log each model request, response, and fallback event as JSON lines. This enables post-run analysis of AI behavior, response validity, and policy rationale. Trace logging is separate from mission logging so that control events and model-interaction events can be analyzed independently.

VI. BACKEND AND COMMUNICATION LAYER

A. Backend Modes

The backend abstraction supports progressive development from offline testing to live digital twin operation. Table II summarizes the current backend modes.

The mock backend is not intended to replace the Simulink model. It exists so the runner, policy interface, guard layer, command limiting, mission status logic, and report generation can be tested before live model integration.

The MATLAB/Simulink backend uses MATLAB Engine to initialize the model, apply command variables, step the

simulation, and extract observations. The bridge expects command variables such as `P_batt_cmd`, `enable_cmd`, and `mode_cmd`. Observations are returned as JSON to avoid fragile MATLAB-to-Python struct conversion.

The live UDP backend supports a continuously running Simulink model. In this mode, Simulink sends observation packets to the Python runner and receives command packets from it. The runner does not call MATLAB Engine to advance the simulation; instead, it waits for live observation packets and resends the latest command at a configurable interval.

B. UDP Protocol

The UDP packet schema is compact JSON with a protocol name, version, packet kind, sequence number, timestamp, and command or observation payload. Commands include both string modes and numeric mode codes:

$$\begin{aligned} \text{CHARGE} &= -1, & \text{HOLD} &= 0, \\ \text{DISCHARGE} &= 1, & \text{SAFE} &= 99. \end{aligned} \quad (8)$$

The Simulink command receiver can include a stale-command timeout. If no fresh command arrives within the allowed time, the model can disable the command path and enter a safe mode. This creates a second safety layer inside the digital twin runtime.

VII. CASE STUDY: BATTERY-ASSISTED DC BUS

A. System Description

The framework is demonstrated using a battery-assisted dc load-bus digital twin. The digital twin includes a dc bus, source, load, battery, and converter-level control path. The autonomous supervisor is responsible for maintaining SOC within a target band while keeping bus voltage within a desired range. The supervisor does not directly control the converter duty ratio. Instead, it sends a battery-power command to the digital twin, and the model's internal controller maps that command to converter behavior.

This case study is representative of a broader class of power-system digital twin problems in which supervisory decisions must coordinate energy storage, load support, source limits, and safety constraints. The same framework can also be mapped to a laboratory dc microgrid with two parallel converter-interfaced sources, droop-based load sharing, a common dc bus, and a variable resistive load. In that setting, the observation and command contracts would be extended to include source currents, droop coefficients, load-sharing ratios, and bus-voltage regulation objectives.

B. Implementation Summary

The implementation includes the following artifacts:

- `AutonomousRunner`, which implements the guarded supervisory loop;
- `DTObservation`, `DTCommand`, and `DTGoal`, which define the state-action-goal contract;
- `SocBandPolicy`, which provides a deterministic baseline policy;

- `LLMControlPolicy`, which provides an OpenAI-compatible AI policy;
- `MockBatteryBusBackend`, `SimulinkBackend`, and `UdpLiveSimulinkBackend`, which support multiple runtime modes; and
- reporting utilities that generate CSV and Markdown audit logs.

The initial mission objective is intentionally modest: maintain SOC inside the target band, maintain bus voltage inside the soft bus-voltage band, stop safely if SOC or bus voltage crosses hard limits, and record each supervisory decision and constraint violation. This objective validates the framework before introducing more complex optimization objectives such as energy management, adaptive droop coordination, or predictive scheduling.

C. Mission Evidence

The current prototype produces mission reports containing mission status, mission reason, number of supervisor decisions, simulation time range, SOC start and end values, bus-voltage start and end values, telemetry extrema, commanded-power extrema, mode counts, constraint violations, and a decision timeline. This format provides a direct audit trail for each run. For example, nominal mock-backend runs show repeated `HOLD` decisions when SOC and bus voltage are inside the target band. Hard-limit scenarios show deterministic transition to `SAFE` mode. Live UDP trials demonstrate that the same runner can interact with a continuously running digital twin through a packet interface.

Because this is a framework paper, these demonstrations support feasibility, modularity, guard behavior, and auditability rather than claiming universal control optimality.

VIII. EVALUATION PLAN

The evaluation should answer four framework-level questions: (i) whether the framework preserves safety constraints independently of the active policy; (ii) whether different policies can be substituted through the same interface; (iii) whether the same runner can operate over mock, step-based Simulink, and live UDP backends; and (iv) whether the reporting layer produces enough evidence to audit each decision.

Metrics should include task success rate, hard-constraint violation count, soft-constraint violation count and duration, safe-stop correctness, command validity rate, command clipping frequency, command rate-limit activation frequency, AI fallback rate, communication timeout count, decision latency, audit-log completeness, and backend portability across scenarios. These metrics are aligned with the framework claim: the architecture should capture, bound, and report each run consistently across policies and backends.

Recommended comparisons include deterministic baseline policy with guard layer, AI policy with guard layer, AI policy in strict mode, guarded policy with command rate limiting disabled in offline simulation, mock backend versus Simulink backend, and step-based Simulink bridge versus live UDP communication. An unguarded AI policy should not be run

TABLE III
PLANNED SCENARIO MATRIX FOR THE FINAL FRAMEWORK EVALUATION.

Scenario	Initial condition or disturbance	Expected framework behavior	Primary metric
Nominal in-band operation	SOC and bus voltage inside target bands	Hold command, no violations	Correct hold behavior
Low SOC with source available	SOC below target, source available	Charge command subject to limits	Correct mode and bounded power
Low bus voltage	Bus voltage below soft band, SOC above reserve	Discharge for bus support	Voltage-support response
Hard SOC violation	SOC at or beyond hard reserve or maximum	Immediate safe stop	Safety-stop correctness
Communication interruption	Missing or stale UDP observations or commands	Timeout, fallback, or safe behavior	Communication robustness
AI malformed output	Invalid JSON or invalid mode from policy	Fallback or strict failure	Policy-failure handling
Backend substitution	Same scenario across mock and Simulink backends	Same runner and policy interface	Backend portability

against a live or physical system; if included, it should be restricted to offline simulation to illustrate the value of guard enforcement.

IX. DISCUSSION

The proposed framework addresses a gap between digital twin modeling and autonomous control. Many digital twin architectures emphasize representation, synchronization, and visualization. Many control architectures emphasize plant-level control design. The proposed framework focuses on the layer between them: the guarded autonomous supervisor that treats the digital twin as an executable runtime while preserving explicit safety and audit contracts.

The framework is especially useful when experimenting with AI-based policies. Language models and other learned policies may offer flexible reasoning, natural-language rationales, or adaptation to structured context. However, they should not be treated as safety-critical controllers. In this framework, the AI policy is an interchangeable proposal generator. It can suggest a high-level command, but the runner checks constraints, clips commands, rate-limits power, and decides whether to continue or stop.

The backend abstraction also improves development workflow. Researchers can begin with a deterministic mock backend to validate runner logic and scenario definitions. They can then use MATLAB/Simulink engine integration for controlled step-based simulation. Finally, they can move to live UDP communication when the digital twin is running continuously. This progression reduces the risk of debugging AI behavior, communication behavior, and plant-model behavior simultaneously.

X. LIMITATIONS AND FUTURE WORK

The present framework has several limitations. First, the case-study objective is intentionally simple. Maintaining SOC and bus-voltage bands demonstrates the architecture but does not fully exploit predictive digital twin capabilities. Future work should include forecast-aware energy management, adaptive load sharing, and multi-objective optimization. Second,

the current command space is compact. More complex microgrid scenarios may require commands for droop coefficients, source dispatch, load shedding, operating-mode transitions, or fault-isolation decisions. Third, the current implementation focuses on supervisory control in simulation and live digital twin environments. Physical hardware integration would require additional safety certification, real-time guarantees, hardware interlocks, and operator approval pathways. Fourth, AI-policy evaluation remains preliminary. Final claims about AI control quality require a larger scenario set, controlled baselines, repeated trials, and analysis of malformed outputs, latency, and fallback frequency.

Future work will extend the framework in four directions: a richer microgrid case study with adaptive droop and load-sharing control; scenario libraries for repeatable autonomous digital twin benchmarking; formal policy-safety contracts for learned and LLM-based supervisors; and integration with operator-in-the-loop approval and explanation workflows.

XI. CONCLUSION

This paper proposed a modular framework for guarded autonomous digital twin supervisory control. The framework defines a structured observation, goal, command, policy, backend, guard, and reporting architecture. It separates high-level policy proposals from deterministic safety enforcement and backend execution. It supports deterministic and AI-based policies through a common interface, and it can operate over mock simulation, MATLAB/Simulink engine control, and live UDP communication. A battery-assisted dc bus digital twin case study demonstrates how the framework can be instantiated for power-system supervisory control while preserving auditability and safety constraints.

The main contribution is not a single controller tuned for one scenario. Rather, the contribution is a reusable architecture for developing, testing, and evaluating autonomous digital twin controllers. By making the state-action contract explicit, keeping safety outside the policy, and recording each decision, the framework provides a practical path from passive digital twins toward guarded autonomous digital twin operation.

ACKNOWLEDGMENT

TODO: Add funding, laboratory, collaborator, and data/model acknowledgments required by the target IEEE special call.

REFERENCES

- [1] TODO: Add a verified foundational digital twin reference in IEEE format.
- [2] TODO: Add a verified digital twin survey reference in IEEE format.
- [3] TODO: Add a verified digital twin reference for power systems or microgrids.
- [4] TODO: Add a verified autonomous or control-oriented digital twin reference.
- [5] TODO: Add a verified guarded autonomy, safety filter, or safe supervisory control reference.
- [6] TODO: Add a verified reference for LLM agents or tool-using policies if the LLM policy remains in the final submission.
- [7] TODO: Add a verified model-based design, MATLAB/Simulink, or co-simulation reference if required by the venue.
- [8] TODO: Add a verified dc microgrid or battery energy storage supervisory control reference.
- [9] TODO: Add prior hierarchical or image-based digital twin work from the local project literature set.
- [10] TODO: Add prior dc microgrid digital twin forecasting or control work from the local project literature set.